

Linked Lists

Linked lists are probably the simplest data structures you'll build. However, some of the concepts you use to build them are also used to build the most sophisticated data structures described in this book. To use a linked list, you need to understand cells and links in addition to methods of finding, inserting, and deleting cells. You use those same concepts to build complicated networks, trees, and balanced trees, which can be extremely confusing.

This chapter explains the ideas you need to master to use linked lists. Later chapters (in particular, Chapters 4, 5, 8, and 10 through 14) revisit these ideas.

Basic Concepts

A *linked list* is built of objects that are often called *cells*. The cell's class contains whatever data the list must store plus a *link* to another cell. The link is simply a reference or pointer to another object of a cell's class. Often, the pointer field in the cell class is called `Next`.

For example, the following code shows the definition of an `IntegerCell` class in C#. The cell holds an integer value and a pointer to the next `IntegerCell` object in the linked list.

```

class IntegerCell
{
    public int Value;
    public IntegerCell Next;
}

```

Linked lists are often represented graphically, with boxes representing cells and arrows representing links.

To indicate a link that doesn't point to anything, I use a small box with an X in it. (In a programming language, the value of the pointer corresponding to the link would be nothing, null, none, or some other language-specific value indicating that the pointer doesn't point to anything.)

In addition to the list itself, a program needs a variable that points to the list so that the code can find it. Often, this variable is named `top` to indicate that it represents the top of the list. The `top` variable could be a variable of the cell's class, or it might be a pointer to the first cell in the list.

Figure 3.1 shows two linked lists holding the numbers 31, 72, 47, and 9. In the list on the top, the program has a variable named `top` that is a pointer to the list's first cell. In the list on the bottom, the program's `top` variable is the first cell in the list. Both lists end with a box containing an X to represent a null pointer.

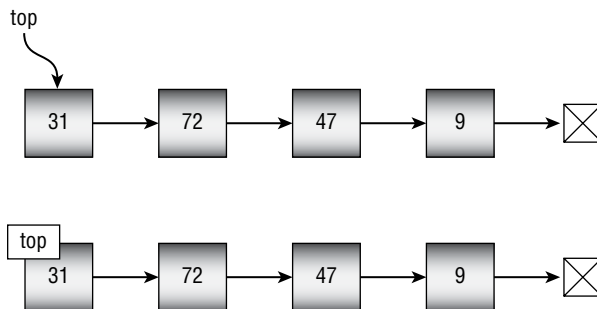


Figure 3.1: These linked lists hold the numbers 31, 72, 47, and 9.

Linked lists are a good way to store a list of items that can grow or shrink over time. To add a new cell, you just add it at the beginning or end of the linked list. In contrast, an array has a fixed size, so it may be hard to enlarge if you need to add more items.

The following sections explain some of the algorithms that you can use to manipulate linked lists. Many of them are most easily described with figures that show the list before and after an operation has been performed.

Singly Linked Lists

In a *singly linked list*, each cell is connected to the following cell by a single link. The lists shown in Figure 3.1 are singly linked lists.

To use a linked list, you need a set of algorithms for iterating over a list, adding items to the list, finding items in the list, and removing items from the list. The following sections describe some of the algorithms that you might want to use.

Iterating Over the List

Assuming that a program has built a linked list, iterating over its cells is relatively easy. The following algorithm shows how you can iterate over the cells in a list and use some sort of method to do something with the values in the cells. This example uses a `Print` method to display the cells' values, but you could replace `Print` with any method that does something with the cells.

```
Iterate(Cell: top)
    While (top != null)
        Print (top.Value)
        top = top.Next
    End While
End Iterate
```

NOTE These algorithms assume that the parameter `top` is passed by value, so the code can modify it without changing the value of `top` in the calling code.

This algorithm starts with a `While` loop that executes as long as the `top` cell pointer is not `null`. Inside the loop, the algorithm calls the `Print` method to display the `top` cell's value. It then sets `top` to point to the next cell in the linked list.

This process continues until `top` is set to the `null` pointer at the end of the list and the `While` loop stops.

This algorithm examines every cell in the linked list, so if the list contains N cells, it has run time $O(N)$.

Finding Cells

Finding a cell in a linked list is simply a matter of iterating over the list and stopping when you find the cell you want. The following algorithm looks through a list and returns the cell that contains a target value:

```
Cell: FindCell(Cell: top, Value: target)
    While (top != null)
        If (top.Value == target) Then Return top
        top = top.Next
    End While

    // If we get this far, the target is not in the list.
    Return null
End FindCell
```

The algorithm enters a `While` loop that executes as long as `top` is not `null`. Inside the loop, the algorithm compares the `top` cell's value to the target value. If the values match, the algorithm returns `top`. If the values do not match, the algorithm moves `top` to point to the next cell in the list.

If `top` runs all the way through the list and becomes `null`, then the target value is not in the list, so the algorithm returns `null`. (Alternatively, the algorithm could throw an exception or raise some kind of error, depending on your programming language.)

As you'll see in some of the following sections, it's often easiest to work with a cell in a linked list if you have a pointer to the cell *before* that cell. The following algorithm finds the cell before the cell that contains a target cell:

```
Cell: FindCellBefore(Cell: top, Value: target)
    // If the list is empty, the target value isn't present.
    If (top == null) Return null

    // Search for the target value.
    While (top.Next != null)
        If (top.Next.Value == target) Then Return top
        top = top.Next
    End While

    // If we get this far, the target is not in the list.
    Return null
End FindCellBefore
```

This code is similar to the previous version—with two exceptions. First it must check that `top` is not `null` before it starts so that it knows it can look at `top.Next` safely. If `top` is `null`, then `top.Next` is undefined, and a program that implemented the algorithm would fail and probably crash.

If `top` is not `null`, the algorithm enters a `While` loop as before, but this time it looks at `top.Next.Value` instead of `top.Value`. When it finds the value, `top` points to the cell before the one that holds the target value, and the algorithm returns `top`.

Using Sentinels

If you study the preceding algorithm closely, you'll find one situation where it fails. If the first cell in the linked list contains the target value, then there is no cell before that one, so the algorithm cannot return it. The first value that the algorithm examines is in the list's second cell, and the algorithm never looks back.

One way to handle this situation is to add special-purpose code that explicitly looks for the target value in the first cell and then handles that case separately. The program would probably need to handle this situation as a special case, and it could get messy.

Another approach is to create a sentinel at the beginning of the list. A *sentinel* is a cell that is part of the linked list but that doesn't contain any meaningful data. It is used only as a placeholder so that algorithms can refer to a cell that comes before the first cell.

The following pseudocode shows the previous FindCellBefore algorithm modified to use a sentinel:

```
Cell: FindCellBefore(Cell: top, Value: target)
    // Search for the target value.
    While (top.Next != null)
        If (top.Next.Value == target) Then Return top
        top = top.Next
    End While

    // If we get this far, the target is not in the list.
    Return null
End FindCellBefore
```

This version doesn't need to check whether `top` is `null`. Because the linked list always has at least a sentinel, `top` cannot be `null`. This means that the `while` loop can begin right away.

This version also starts by checking the value in the first cell in the list, not the second, so it can detect the case where the first cell contains the target value.

This version of the algorithm can return the sentinel cell before the first real cell if appropriate. Therefore, the program using the algorithm doesn't need customized code to handle the special case in which the target value is at the beginning of the list.

When searching for a target value, the algorithm might get lucky and find it right away. But in the worst case it may have to search most of the linked list before it finds the target value. If the target value isn't in the list, the algorithm needs to search every cell in the list. If the list contains N cells, that means this algorithm has run time $O(N)$.

A sentinel may seem like a waste of space, but it removes the need for special-purpose code and makes the algorithm simpler and more elegant.

The following sections assume that linked lists have sentinels.

Adding Cells at the Beginning

One use for linked lists is to provide a data structure where you can store items. This is sort of like an array that you can expand whenever you need more space.

The easiest way to add an item to a linked list is to place a new cell at the beginning, right after the sentinel. The following algorithm adds a new item at the beginning of a linked list:

```
AddAtBeginning(Cell: sentinel, Cell: new_cell)
    new_cell.Next = sentinel.Next
    sentinel.Next = new_cell
End AddAtBeginning
```

This algorithm sets the new cell's `Next` pointer so that it points to the first cell in the list after the sentinel. It then sets the sentinel's `Next` pointer to point to the new cell. That places the new cell after the sentinel so that it becomes the new first cell in the linked list.

Figure 3.2 shows a linked list before and after a new cell is added at the top of the list.

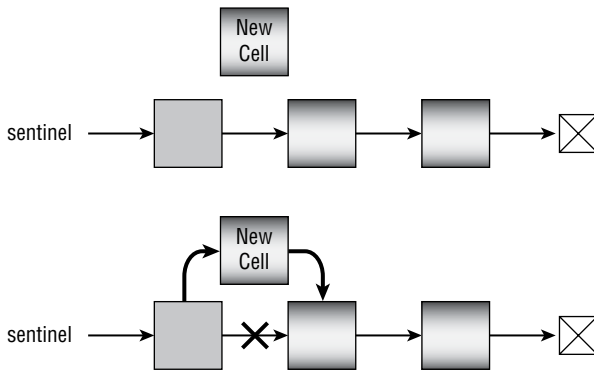


Figure 3.2: To add an item at the top of a linked list, make the new cell's `Next` link point to the old top of the list and then make the sentinel's `Next` link point to the new cell.

This algorithm performs only two steps, so its run time is $O(1)$ no matter how many cells the list contains.

Adding Cells at the End

Adding a cell at the end of the list is a bit more difficult than adding it at the beginning because the algorithm must first traverse the list to find the last cell.

The following pseudocode shows an algorithm for adding a new cell at the end of a list:

```
AddAtEnd(Cell: sentinel, Cell: new_cell)
    // Find the last cell.
    While (sentinel.Next != null)
        sentinel = sentinel.Next
    End While
```

```
// Add the new cell at the end.
sentinel.Next = new_cell
new_cell.Next = null
End AddAtEnd
```

The code iterates through the linked list until it finds the last cell. It makes the last cell's `Next` link point to the new cell and then sets the new cell's `Next` link to point to `null`.

This code would be messier if the list didn't have a sentinel. In that case, you would have to use special code to handle the case when the list is empty so `sentinel` points to `null`.

Figure 3.3 shows the process graphically.

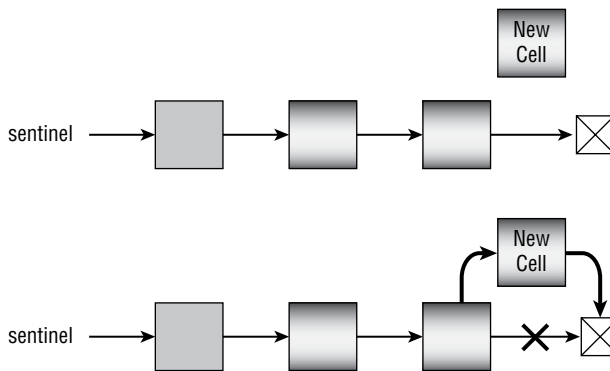


Figure 3.3: To add an item at the end of a linked list, find the last cell and make its `Next` link point to the new cell. Then make the new cell's `Next` link point to `null`.

This algorithm must traverse the entire list, so if the list contains N cells, it has run time $O(N)$.

Inserting Cells After Other Cells

The preceding sections explained how you can add cells at the beginning or end of a linked list, but sometimes you may want to insert an item in the middle of the list. Assuming you have a variable named `after_me` that points to the cell after which you want to insert the item, the following pseudocode shows an algorithm for inserting a cell after `after_me`:

```
InsertCell(Cell: after_me, Cell: new_cell)
    new_cell.Next = after_me.Next
    after_me.Next = new_cell
End InsertCell
```

This algorithm makes the new cell's `Next` link point to the cell that follows `after_me` and then makes `after_me`'s `Next` link point to the new cell. Figure 3.4 shows the process graphically.

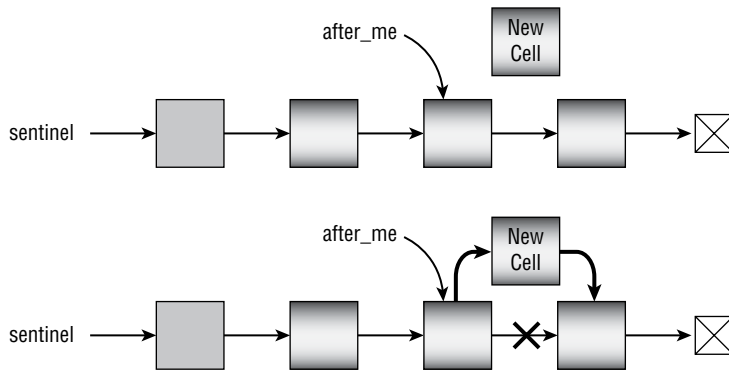


Figure 3.4: Inserting a cell after a given cell takes $O(1)$ time.

This algorithm takes only two steps, so it runs in $O(1)$ time, although you may need to use $O(N)$ time to find the cell `after_me`. For example, if you want to insert a cell after the cell that contains a target value, first you need to find the cell that contains the target value.

Deleting Cells

To delete a target cell, you simply set the previous cell's `Next` link to the cell that follows the target cell. The following pseudocode shows an algorithm that deletes the cell after the cell `after_me`:

```

DeleteAfter(Cell: after_me)
    Cell: target = after_me.Next
    after_me.Next = target.Next
End DeleteAfter

```

Figure 3.5 shows this algorithm graphically.

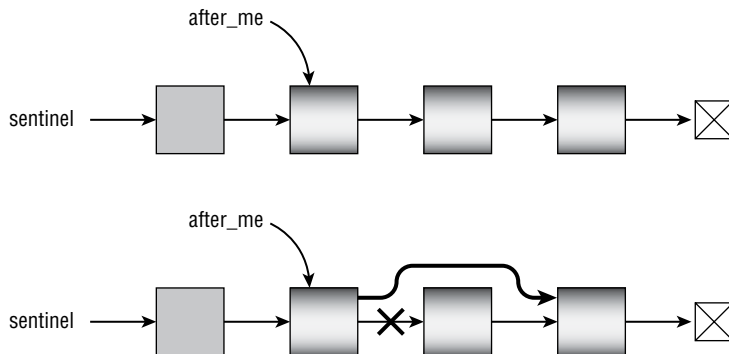


Figure 3.5: To remove a cell from a linked list, set the previous cell's `Next` link to point to the cell after the target cell.

Programming languages such as Python, C#, and Visual Basic use automatic memory management, so the deleted cell is eventually recycled automatically. In some other programming languages, such as C and C++, you may need to perform extra work to free the deleted cell properly. For example, in C++ you would need to free the target cell, as in the following version of the algorithm:

```
DeleteAfter(Cell: after_me)
    Cell: target_cell = after_me.Next
    after_me.Next = after_me.Next.Next
    free(target_cell)
End DeleteAfter
```

How you destroy a linked list also depends on your language. In Python, C#, and Visual Basic, you can simply set all of the program's references to the list to null, and the garbage collector eventually reclaims the list. In a language such as C++, where you need to free each cell explicitly, you need to walk through the list, freeing each cell, as shown in the following pseudocode:

```
DestroyList(Cell: sentinel)
    While (sentinel != null)
        // Save a pointer to the next cell.
        Cell: next_cell = sentinel.Next

        // Free sentinel.
        free(sentinel)

        // Move to the next cell.
        sentinel = next_cell
    End While
End DestroyList
```

How you free resources is language-dependent, so this book doesn't say anything more about it here or in later chapters. Just be aware that you may need to do some extra work whenever you remove a cell or other object from a data structure.

Doubly Linked Lists

In a *doubly linked list*, the cells have links that point to both the next and previous cells in the list. The link to the previous cell is often called `Prev` or `Previous`.

Often, it is convenient to have both top and bottom sentinels for doubly linked lists so that the program can easily manipulate the list from either end. For example, this lets you add items to and remove items from either end in $O(1)$ time.

Figure 3.6 shows a doubly linked list with top and bottom sentinels.

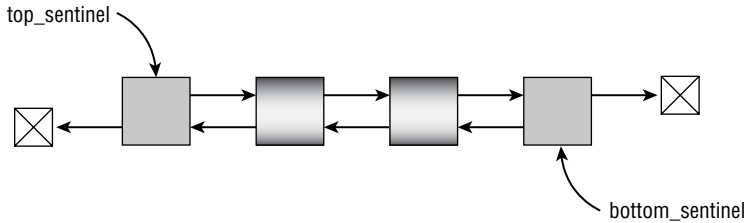


Figure 3.6: Doubly linked lists often have top and bottom sentinels.

Algorithms for manipulating doubly linked lists are similar to those that work with singly linked lists, except that they must do some extra work to manage the second set of links. For example, the following pseudocode shows an algorithm for inserting a cell after a given cell:

```

InsertCell(Cell: after_me, Cell: new_cell)
    // Update Next links.
    new_cell.Next = after_me.Next
    after_me.Next = new_cell

    // Update Prev links.
    new_cell.Next.Prev = new_cell
    new_cell.Prev = after_me
End InsertCell
  
```

The only really tricky part of these algorithms is keeping track of which links have been updated at any point in time. For example, in the preceding algorithm, the second-to-last statement sets the `Prev` link that should point to the new cell. You might be tempted to do this by using the following statement:

```
after_me.Next.Prev = new_cell
```

However, when this statement executes, `after_me.Next` has already been updated to point to the new cell, so this won't work. The algorithm needs to use `new_cell.Next` instead.

Figure 3.7 shows the algorithm graphically.

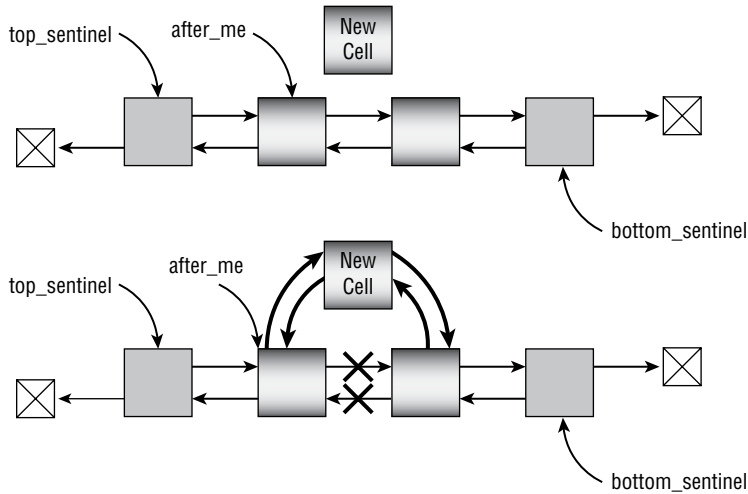


Figure 3.7: When updating a doubly linked list, a program must update both the `Next` and `Prev` links.

Sorted Linked Lists

Sometimes, it's convenient to keep the items in a linked list in sorted order. When you add a new item to the list, you need to search through the list to find the position where the item belongs and update the appropriate links to insert it there.

The following pseudocode shows an algorithm for inserting an item into a sorted singly linked list:

```
// Insert a cell into a sorted singly linked list.
InsertCell(Cell: sentinel, Cell: new_cell)
    // Find the cell before where the new cell belongs.
    While (sentinel.Next != null) And
        (sentinel.Next.Value < new_cell.Value)
        sentinel = sentinel.Next
    End While

    // Insert the new cell after sentinel.
    new_cell.Next = sentinel.Next
    sentinel.Next = new_cell
End InsertCell
```

In the worst case, this algorithm might need to cross the whole list before finding the correct location for the new item. Therefore, if the list holds N cells, its run time is $O(N)$.

Although you cannot improve the theoretical run time, you can make the algorithm simpler and slightly faster in practice by adding a bottom sentinel. If you set the bottom sentinel's `Value` to a value larger than any `Value` that could be stored in a cell, then you can remove the `sentinel.Next != null` test because you know that the code will eventually find a location for the new cell, even if it's right before the bottom sentinel.

For example, if the cells hold names that use ASCII characters, you can set the bottom sentinel's `Value` to `~` because the `~` character comes alphabetically after any valid name. If the cells hold integers, you can set the bottom sentinel's `Value` to the largest possible integer value. (On most 32-bit systems, that value is 2,147,483,647. Python can represent arbitrarily large integers, so you'll just have to pick a value that is larger than any value that you might want to store in the list.)

The following pseudocode shows the revised algorithm, assuming that the list has a bottom sentinel holding a value larger than any value that could be held in the cells:

```
// Insert a cell into a sorted singly linked list.
InsertCell(Cell: sentinel, Cell: new_cell)
    // Find the cell before where the new cell belongs.
    While (sentinel.Next.Value < new_cell.Value)
        sentinel = sentinel.Next
    End While

    // Insert the new cell after sentinel.
    new_cell.Next = sentinel.Next
    sentinel.Next = new_cell
End InsertCell
```

Self-Organizing Linked Lists

A *self-organizing linked list* is a list that uses some sort of heuristic to rearrange its items to improve expected access times. For example, if the program searches the list for a specific item a large number of times, it would make sense to move that item to the beginning of the list so that it is easier to find.

NOTE A *heuristic* is an algorithm that is likely but not guaranteed to produce a good result. For example, a heuristic for not getting speeding tickets is to drive no more than 5 miles per hour over the speed limit. You still *might* get a ticket, but you probably won't. (But don't take my word as legal advice! Don't blame me if you get a ticket.) You'll learn a lot more about heuristic algorithms later in this book.

If you number the items in the list $1, 2, \dots, N$, and the probability that you need to find item i is P_i , then the expected number of steps that you need to move down the list to find an item is given by the following formula:

$$\text{Expected Search Length} = 1 \cdot P_1 + 2 \cdot P_2 + 3 \cdot P_3 + \dots + N \cdot P_N$$

If it is equally likely that you'll want to find any given item, then all of the probabilities P_i have the same value $1 / N$ and the expected search length is the following:

$$\begin{aligned} \text{Expected Search Length} &= \frac{1}{N} + \frac{2}{N} + \frac{3}{N} + \dots + \frac{N}{N} = \\ \frac{1}{N} \sum i &= \frac{N(N+1)}{2N} = \frac{N+1}{2} \end{aligned}$$

This value depends only on the length of the list, so it doesn't matter how the items are arranged. Because the P_i values are all the same, you will need to search roughly halfway through the list on average to find any given item.

In contrast, if the P_i values are different, then it makes sense to move the more popular items to the front of the list. If you knew the P_i values ahead of time, then you could arrange the list optimally. Unfortunately, you usually won't know the actual probabilities until after you perform some searches.

A self-organizing list rearranges its items as it goes along to try to improve the order of its items. The following sections describe several ways that a self-organizing list can rearrange its items to improve search times.

Move To Front (MTF)

In the *move to front method*, the list moves the most recently accessed item to the front of the list. Moving an item to the front of a link list only takes a few steps, so this is relatively fast and easy. Frequently accessed items will tend to remain near the top of the list while those that are accessed less often will usually be near the bottom of the list.

One drawback to this method is that an infrequently accessed item will occasionally jump to the front of the list and slow down subsequent searches until it is pushed back down the list. Still, this kind of list gives a significant improvement for little extra work.

Swap

The *swap method* or *transpose method* swaps the most recently accessed item with the item before it so that frequently accessed items gradually move toward the front of the list. It takes only a few steps to swap two items in a linked list, so this method is fast and easy. It also prevents an infrequently accessed item from jumping to the front of the list and slowing down later searches.

One drawback to this method is that items move slowly. That means frequently accessed items can take a long time to move to the front of the list, so it can take a while before the items reach an effective arrangement. In turn, that means the MTF method may give better results if you are only going to perform a smaller number of searches.

Count

In the *count method*, you keep track of the number of times each item is accessed. When you search for an item, you increment its count and then move it up in the list until it lies before any items that have smaller counts. Over time, the items move close to their optimal arrangement where they are sorted by their probabilities.

One drawback to this method is that it requires extra storage to hold the item counts. It also takes more work to move an item several positions through the list than it does simply to move it to the front of the list or to swap it with one neighboring item.

Despite the fact that it may take a relatively large amount of work to rearrange the list, the items move into near-optimal positions fairly quickly, and after that, the items should require little rearrangement.

Hybrid Methods

The MTF, swap, and count methods have different behaviors during different parts of the list's lifetime. For example, the MTF method makes large adjustments to the items' order relatively quickly, but later searches for less commonly accessed items can mess up the arrangement. Swapping produces a better arrangement but is slower. Counting produces a very good arrangement but requires extra storage.

You may be able to use a combination of techniques to produce a better overall result. For example, you might use an MTF strategy to move the most commonly accessed items quickly to the front part of the list. Then you could switch to a swapping strategy to adjust the list more slowly.

Another approach might initially use MTF while updating item counts. After performing enough searches to get useful statistics, you could sort the items by their counts and then switch to a counting strategy.

Figure 3.8 shows the `SelfOrganizingLists` example program after it has searched a list of 100 values 1 million times. You can see that the MTF list was much faster than the nonarranging list, the swapping list was even faster, and the counting list was the fastest of all.

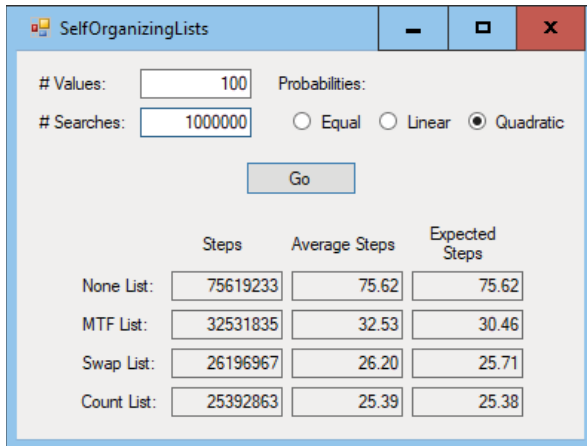


Figure 3.8: Self-arranging lists are much faster than fixed lists.

The program's radio buttons determine how the items' probabilities are distributed. If you check the Equal button, all of the items are selected with equal probability.

If you check the Linear button, then an item's probability is proportional to its value. For example, the value 10 is selected 10 times as often as the value 1.

If you check the Quadratic button, an item is selected with probability equal to its square divided by the total of all of the numbers squared. For example, the value 17 is selected with probability $17 \times 17 / \text{total}$.

Pseudocode

You can use the following high-level pseudocode for any self-organizing list:

```
// Find an item and rearrange the linked list.
Item: FindItem(Value: value)
    <Find the item.>
    <Rearrange the list for the desired self-organizing strategy.>
    <Return the item.>
End FindItem
```

You can use the techniques described earlier in this chapter to find the item.

For an MTF or swapping list, you can find the linked-list cell before the target item so that you can move or swap the item more easily.

For a counting list, you can use a doubly linked list so that you can easily swap the found item toward the front of the list as far as necessary.

To implement the algorithm in C#, Python, or another object-oriented language, you can make a self-organizing list parent class that provides basic list methods to add and remove items. Its `Find` method can call a `Rearrange` method that does

nothing in the parent class. Then the MTF, swapping, and counting child classes can override the `Rearrange` method to restructure the list appropriately. That allows the parent class to hold as much shared code as possible, and the child classes only need to contain code that is specific to their arranging strategies.

Linked-List Algorithms

So far, this chapter has described algorithms for building and maintaining linked lists. It has described algorithms for adding items at the top, bottom, and interior of a list; algorithms for finding and deleting items in a list; and algorithms for making self-organizing lists.

The following sections describe other algorithms that manipulate linked lists in other ways.

Copying Lists

Some algorithms rearrange a list. For example, the next two sections describe algorithms that sort the items in a list. If you want to keep the original list intact, you must make a copy of the list before you sort it.

The following pseudocode shows how you can copy a singly linked list:

```
// Copy a list.
Cell: CopyList(Cell: old_sentinel)
    // Make the new list's sentinel.
    Cell: new_sentinel = New Cell()

    // Keep track of the last item we've added so far.
    Cell: last_added = new_sentinel

    // Skip the sentinel.
    Cell: old_cell = old_sentinel.Next

    // Copy items.
    While (old_cell != null)
        // Make a new item.
        last_added.Next = New Cell

        // Move to the new item.
        last_added = last_added.Next

        // Set the new item's value.
        last_added.Value = old_cell.Value

        // Get ready to copy the next cell.
        old_cell = old_cell.Next
    End While
```



```

    // End with null.
    last_added.Next = null

    // Return the new list's sentinel.
    Return new_sentinel
}

```

This algorithm is reasonably straightforward, but it contains one feature worth mentioning. The algorithm uses the variable `last_added` to keep track of the cell that was most recently added to the new copy of the list. To copy a new item to the list, the algorithm sets `last_added.Next` equal to a new cell object. That puts the new object at the end of the list. The algorithm then updates `last_added` to point to the new item and copies the original cell's value into it.

This lets the list grow at the bottom instead of at the top. This is similar to how you can easily add items to the end of a list if you keep track of the last item in the list, as described in Exercise 1.

Sorting with Insertionsort

Chapter 6, “Sorting,” says a lot about sorting algorithms, but two are worth discussing here: `insertionsort` and `selectionsort`.

The basic idea behind `insertionsort` is to take an item from the input list and insert it into the proper position in a sorted output list (which initially starts empty).

The following pseudocode shows the `insertionsort` algorithm, where the items to sort are stored in a singly linked list that has a top sentinel:

```

// Use insertionsort to sort the list.
Cell: Insertionsort(Cell: old_sentinel)
    // Make a sentinel for the sorted list.
    Cell new_sentinel = New Cell()
    new_sentinel.Next = null

    // Skip the input list's sentinel.
    old_sentinel = old_sentinel.Next

    // Repeat until we have inserted all of the items in the new list.
    While (old_sentinel != null)
        // Get the next cell to add to the list.
        Cell: next_cell = old_sentinel

        // Advance old_sentinel for the next trip through the loop.
        old_sentinel = old_sentinel.Next

        // See where to add the next item in the sorted list.
        Cell: after_me = new_sentinel
        While (after_me.Next != null) And

```

```
        (after_me.Next.Value < next_cell.Value)
        after_me = after_me.Next
    End While

    // Insert the item in the sorted list.
    next_cell.Next = after_me.Next
    after_me.Next = next_cell
End While

// Return the sorted list.
return new_sentinel
End Insertionsort
```

This algorithm starts by building an empty list to hold the sorted output. It then loops through the unsorted list of input cells. For each input cell, it looks through the growing sorted list and finds the cell after which the new value belongs. It then inserts the cell there.

You can simplify the code if you call the `InsertCell` algorithm described in the earlier section “Inserting Cells After Other Cells.”

The algorithm’s best case occurs if the items in the input list are initially sorted in largest-to-smallest order. In that case, each time the algorithm considers a new item, it is smaller than all of the items that have already been added to the sorted list. That means the new item belongs at the top of the sorted list, so you can insert it in $O(1)$ time. That in turn means you can “sort” the already-sorted items in a total of $O(N)$ time.

The algorithm’s worst case occurs when the items are initially sorted in smallest-to-largest order. In that case, when the algorithm considers an item, it is larger than all the items that have already been moved to the new list, and you need to insert it at the end of the list. That means inserting all of the items takes $1 + 2 + 3 + \dots + N = N \times (N - 1) / 2 = O(N^2)$ steps.

In the average case, with the items initially randomly arranged, the algorithm can insert some items quickly, but others take longer. The result is that the algorithm’s run time is still $O(N^2)$, although in practice it won’t take as long as the worst case.

Many other sorting algorithms take only $O(N \log N)$ time, so this algorithm’s $O(N^2)$ performance is relatively slow. That makes this algorithm ineffective for large lists. However, it runs reasonably quickly for small lists, and it works for linked lists, which many of the other algorithms don’t.

Sorting with Selectionsort

The basic idea behind the selectionsort algorithm is to search the input list for the largest item it contains and then add it to the front of a growing sorted list.

The following pseudocode shows the selectionsort algorithm for a singly linked list holding integers:

```
// Use selectionsort to sort the list.
Cell: Selectionsort(Cell: old_sentinel)
    // Make a new_sentinel for the sorted list.
    Cell: new_sentinel = New Cell
    new_sentinel.Next = null

    // Repeat until the old list is empty.
    While (old_sentinel.Next != null)
        // Find the largest item in the old list.
        // The cell best_after_me will be the cell before
        // the one with the largest value.
        Cell: best_after_me = old_sentinel
        Integer: best_value = best_after_me.Next.Value

        // Start looking with the next item.
        Cell: after_me = old_sentinel.Next
        While (after_me.Next != null)
            If (after_me.Next.Value > best_value) Then
                best_after_me = after_me
                best_value = after_me.Next.Value
            End If
            after_me = after_me.Next
        End While

        // Remove the best cell from the unsorted list.
        Cell: best_cell = best_after_me.Next
        best_after_me.Next = best_cell.Next

        // Add the best cell at the beginning of the sorted list.
        best_cell.Next = new_sentinel.Next
        new_sentinel.Next = best_cell
    End While

    // Return the sorted list.
    Return new_sentinel
End Selectionsort
```

You can simplify this algorithm somewhat if you extract the code that finds the largest cell in the input list, place that code in a new algorithm, and then invoke the new algorithm from this one.

When the input list contains K items, finding the largest item in the list takes K steps. Adding the largest item to the sorted list takes only a few steps. As the algorithm progresses, the input list shrinks. Therefore, if it originally holds N items, the total number of steps is $N + (N - 1) + (N - 2) + \dots + 2 + 1 = N \times (N - 1) / 2 = O(N^2)$, the same run time given by the insertion sort algorithm.

Multithreaded Linked Lists

In a singly linked list, a cell has a link to the next cell in the list. In a doubly linked list, each cell has links to the cell before and after it in the list. The doubly linked list uses two links to provide two different ways to move through the cells it contains: forward or backward.

There's no reason why you can't add other links to a list's cells to provide other ways to move through the cells. For example, suppose you build a `Planet` class to hold information about the solar system's planets. You can give the `Planet` class a field named `NextDistance` that is a link to the `Planet` that is the next nearest to the sun. Following the `NextDistance` links would list the planets in the order Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, and Neptune (and optionally Pluto, if you're a Pluto fan).

Similarly, you could add other fields to list the planets ordered by mass, diameter, and other characteristics. Each path through the cells defined by a set of links is called a *thread*.

It's easy enough to work with a single thread, thinking of it as a simple linked list, although visualizing all of the threads at the same time can be messy. For example, Figure 3.9 shows a linked list of planets with three threads. The thin links visit the planets ordered by distance to the sun, the dashed links visit the planets ordered by mass, and the thick links visit the planets ordered by diameter.

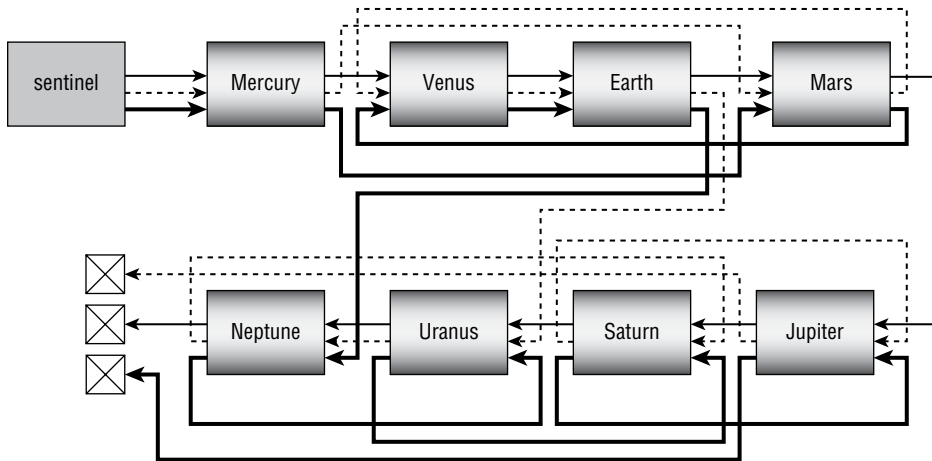


Figure 3.9: Visualizing all the threads through a multithreaded linked list can be confusing.

NOTE Other data structures can also have threads. For example, a tree might provide threads to let a program visit its nodes in orders that are not typical for a tree.

Linked Lists with Loops

A *circular linked list* is a linked list in which the last link points back to the first item in the list. Figure 3.10 shows a circular linked list.

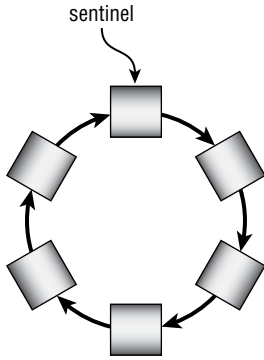


Figure 3.10: Circular linked lists let a program easily loop through a sequence of objects indefinitely.

Circular linked lists can be useful when you need to loop through a sequence of items indefinitely. For example, an operating system might repeatedly loop through a list of processes to give each a chance to execute. If a new process started, it could be added anywhere in the list, perhaps right after the sentinel so that it would have a chance to execute right away.

As another example, a game might loop indefinitely through a list of objects, allowing each to move on the screen. Again, new objects could be added anywhere to the list.

Figure 3.11 shows a linked list that contains a loop, but the loop doesn't include all of the list's cells.

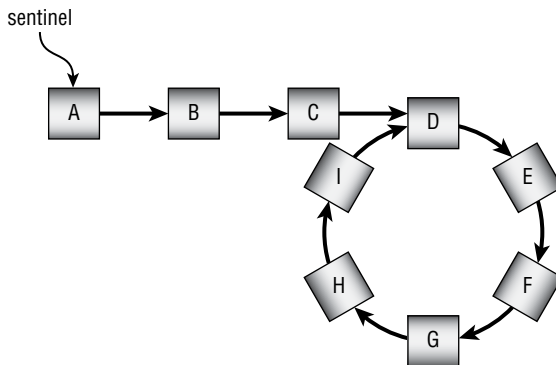


Figure 3.11: This list contains a loop that doesn't include all of the list's cells.

The kind of linked list shown in Figure 3.10 is interesting, mostly because it presents you with two thought-provoking problems. First, how can you tell whether a linked list contains such a loop? Second, if a linked list contains such a loop, how can you find where the loop starts and break it there to “fix” the list? This is roughly the same question as asking where the “bottom” of the list is. In Figure 3.11, you might define the bottom of the list to be cell I because it is the last cell you visit while traversing the list before cells start repeating.

The following sections describe some of the most interesting algorithms that answer these questions.

Marking Cells

Probably the easiest way to tell whether a linked list has a loop is to traverse its cells, marking each as you visit it. If you come to a cell that is already marked, you know that the list has a loop and that it starts at that point.

The following pseudocode shows this algorithm:

```
// Return true if the list has a loop.
// If the list has a loop, break it.
Boolean: HasLoopMarking(Cell: sentinel)
    // Assume there is no loop.
    Boolean: has_loop = false

    // Loop through the list.
    Cell: cell = sentinel
    While (cell.Next != null)
        // See if we already visited the next cell.
        If (cell.Next.Visited)
            // This is the start of a loop.
            // Break the loop.
            cell.Next = null
            has_loop = true
            <Break out of the While loop>
        End If

        // Move to the next cell.
        cell = cell.Next

        // Mark the cell as visited.
        cell.Visited = true
    End While

    // Traverse the list again to clear the Visited flags.
    cell = sentinel
    While (cell.Next != null)
        cell.Visited = false
        cell = cell.Next
    End While
```

```

    // Return the result.
    Return has_loop
End HasLoopMarking

```

The BreakLoopMarking sample program, which is available for download on the book's website, demonstrates this algorithm.

This algorithm must traverse the loop twice—once to set the cells' `visited` flags to `true` and again to reset them to `false`. So, if the list contains N cells, this algorithm takes $2 \times N$ steps and runs in $O(N)$ time.

This algorithm also requires that each cell have an added `visited` field, so it requires $O(N)$ space. The list already takes up $O(N)$ space to hold the cells and their links, so this shouldn't be a problem, but it's worth acknowledging that the algorithm has some memory requirements.

NOTE Marking cells is a simple technique that is also useful for other data structures, particularly networks. Some of the algorithms described in Chapters 13 and 14 use marking techniques.

Often, a problem such as this one has the additional requirement that you are not allowed to change the definition of the cell class. In this case, that means you aren't allowed to add a `visited` field. The following algorithms satisfy that additional restriction.

Using Hash Tables

Hash tables are described in detail in Chapter 8. For now, all you need to know is that a hash table can quickly store items, retrieve items, and tell you whether an item is present in the hash table.

This algorithm moves through the list, adding each cell to a hash table. When it visits a cell, it checks the hash table to see whether the cell is already in the table. If it comes to a cell that is already in the hash table, the list contains a loop that starts at that cell.

The following pseudocode shows this algorithm:

```

// Return true if the list has a loop.
// If the list has a loop, break it.
Boolean: HasLoopHashTable(Cell: sentinel)
    // Make a hash table.
    Hashtable: visited

    // Loop through the list.
    Cell: cell = sentinel
    While (cell.Next != null)
        // See if we already visited the next cell.
        If (visited.Contains(cell.Next))
            // This is the start of a loop.
            // Break the loop and return true.

```

```
        cell.Next = null
        Return true
    End If

    // Add the cell to the hash table.
    visited.Add(cell)

    // Move to the next cell.
    cell = cell.Next
End While

// If we get this far, there is no loop.
Return false
End HasLoopHashTable
```

The BreakLoopHashtable sample program, which is available for download on the book's website, demonstrates this algorithm.

This algorithm traverses the list's cells once, so if the list contains N cells, this algorithm takes N steps and runs in $O(N)$ time.

This algorithm also requires a hash table. For the best performance, a hash table must have extra space beyond what it needs to store the values. If the list contains N items, the hash table must have room for more than N entries. A hash table with room for $1.5 \times N$ entries will give good performance and still use $O(N)$ space.

This algorithm obeys the restriction that it isn't allowed to modify the cell class, but it uses extra storage. The following sections describe some algorithms that detect loops without using extra storage.

List Retracing

The *list retracing algorithm* uses two objects that traverse the list. It starts the first object (call it *leader*) on a normal traversal. Each time that object visits a new cell, the algorithm starts a second object (call it *tracer*) on a traversal. If *tracer* ever reaches the cell *leader.Next* before it comes to the cell *leader*, then the list contains a loop.

For example, look again at the list shown in Figure 3.11, and suppose *leader* is pointing to cell I. Then *tracer* will reach *leader.Next*, which is cell D, before it reaches cell I. That shows there is a loop starting at node D and ending at node I.

The following code shows the algorithm in pseudocode:

```
// Return true if the list has a loop.
// If the list has a loop, break it.
Boolean: HasLoopRetracing(Cell: sentinel)
    // Loop through the list.
    Cell: cell = sentinel
    While (cell.Next != null)
        // See if we already visited the next cell.
        Cell: tracer = sentinel
        While (tracer != cell)
            If (tracer.Next == cell.Next)
```



```

        // This is the start of a loop.
        // Break the loop and return true.
        cell.Next = null
        Return true
    End If
    tracer = tracer.Next
End While

// Move to the next cell.
cell = cell.Next
End While

// If we get here, the list has no loop.
Return false
End HasLoopRetracing

```

The BreakLoopHashtable sample program, which is available for download on the book's website, demonstrates this algorithm.

Assume that the list contains N cells. When the algorithm's `cell` object examines the K th cell in the list, the `tracer` object must traverse the list up to that point, so it must perform K steps. That means the algorithm's total run time is $1 + 2 + 3 + \dots + N = N \times (N - 1) / 2 = O(N^2)$.

This is slower than the previous algorithms, but, unlike those algorithms, the only additional space it requires is for the two cell pointers that traverse the list.

List Reversal

Like the preceding list retracing algorithm, the *list reversal algorithm* uses only a small amount of extra space to hold some pointers. This algorithm, however, works in $O(N)$ time.

This algorithm traverses the list, reversing each cell's `Next` link so that it points to the cell before it in the list instead of the one after it. If the algorithm ever reaches the list's sentinel, then the list contains a loop. If the algorithm reaches a `null` link without reaching the sentinel, then the list doesn't contain a loop.

Of course, moving through the list reversing links messes up the links. To restore them, the algorithm then moves back through the list a second time, reversing the links again so that they point back to where they did originally.

To see how this works, look at the list shown in Figure 3.12.

The top image in Figure 3.12 shows the original list. The algorithm starts at cell A and moves through the list, reversing the links.

The middle image in Figure 3.12 shows the algorithm when it has reached cell I. The links that have been reversed are shown in bold. Next, the algorithm follows the link out of cell I to cell D. It then follows the reversed links from cell D to cells C, B, and A. As it follows those links, the algorithm reverses them again to give the image shown at the bottom of Figure 3.12. Here the links that have been reversed twice are shown with dotted arrows.

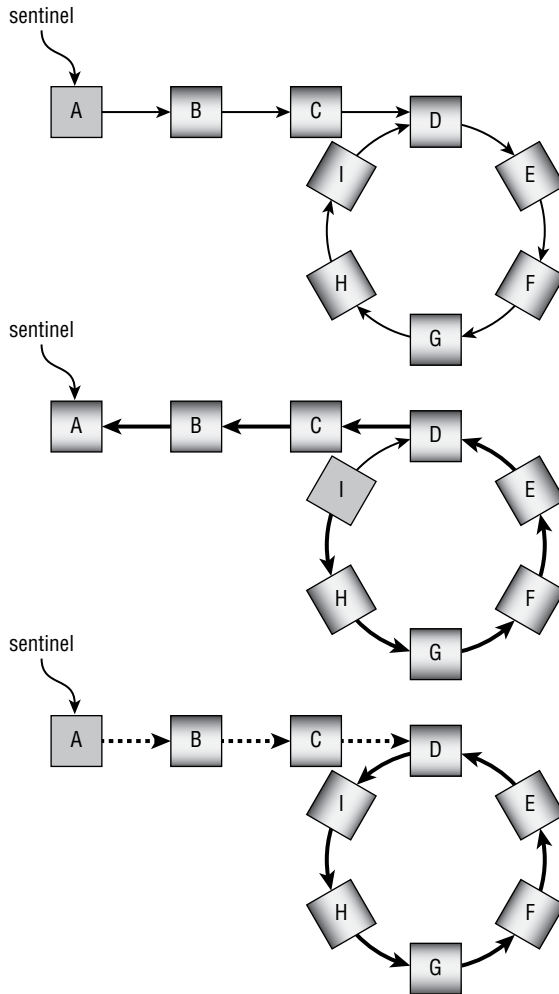


Figure 3.12: An algorithm can detect a loop by reversing the links in a linked list.

At this point, the algorithm returns to the first cell in the list so that it knows the list contains a loop. Notice that the new list is the same as the old one, except that the links in the loop are reversed.

The algorithm finishes by reversing the links again to restore their original links.

Because this algorithm must reverse the list twice, it makes sense to move that operation into a separate method that you can call twice. The following pseudocode shows how the algorithm reverses the list's links:

```
// Reverse the loop once and return the new top of the list.
Cell: ReverseList(Cell: sentinel)
    Cell: prev_cell = null
    Cell: curr_cell = sentinel
```

```

While (curr_cell != null)
    // Reverse the link out of this cell.
    Cell: next_cell = curr_cell.Next
    curr_cell.Next = prev_cell

    // Move to the next cell.
    prev_cell = curr_cell
    curr_cell = next_cell
End While

// Return the last cell we visited.
Return prev_cell
End ReverseList

```

This pseudocode moves through the list, reversing the links, and returns the last node visited, which is the first node in the reversed list.

The following algorithm uses the previous pseudocode to determine whether the list contains a loop:

```

// Return true if the list has a loop.
Boolean: HasLoopReversing(Cell: sentinel)
{
    // If the list is empty, it has no loops.
    If (sentinel.Next == null) Then Return false

    // Loop through the list, reversing links.
    Cell: new_sentinel = ReverseList(sentinel)

    // Loop through the list again to restore the links.
    ReverseList(new_sentinel)

    // If the reversed list starts with the same cell
    // as the original list, there is a loop.
    If (new_sentinel == sentinel) Then Return true
    Return false
End HasLoopReversing

```

This algorithm calls the `ReverseList` method to reverse the list and get the reversed list's first cell. It then calls `ReverseList` again to re-reverse the list and restore the links to their original values.

If the sentinel is the same as the first cell in the reversed list, the algorithm returns `true`. If the sentinel is different from the first cell in the reversed list, the algorithm returns `false`.

This algorithm traverses the list twice—once to reverse links and once to restore them—so it performs $2 \times N = O(N)$ steps.

This algorithm runs in $O(N)$ time without requiring additional space. Unfortunately, it only detects loops; it doesn't provide a way to break them. The next algorithm solves that problem, although it is the most confusing of the algorithms described here.

Tortoise and Hare

This algorithm is called the *tortoise-and-hare algorithm* or *Floyd's cycle-finding algorithm* after Robert Floyd, who invented it in the 1960s. The algorithm itself isn't too complicated, but its explanation is pretty confusing, so if you don't want to see the math, skip to the following pseudocode.

The algorithm starts two objects called `tortoise` and `hare` moving through the list at different speeds starting at the beginning of the list. The tortoise moves one cell per step. The hare moves two cells per step.

If the hare reaches a link that is `null`, then the list has an end, so there is no loop.

If the list does contain a loop, the hare eventually enters the loop and starts running laps around it.

Meanwhile, the tortoise plods along until it eventually reaches the loop, too. At that point, both the tortoise and hare are inside the loop.

Let T be the number of steps that pass before the tortoise enters the loop, and let H be the distance from the beginning of the loop to the hare's location after T steps, as shown in Figure 3.13. Let L be the number of cells inside the loop.

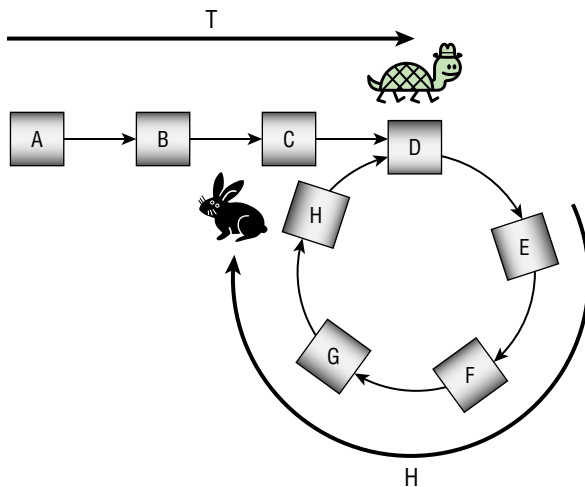


Figure 3.13: T is the distance the tortoise travels to get to the loop, and H is the distance from the start of the loop to the hare at that time.

In Figure 3.13, $T = 4$, $H = 4$, and $L = 5$.

Because the hare moves twice as fast as the tortoise, it reaches the loop after moving T cells. It then crosses T more cells inside the loop to reach the position shown in Figure 3.13. This leads to the following Important Fact #1.

IMPORTANT FACT #1

If you move across T cells within the loop, you end up H cells away from where you started.

Note that the hare may have run several laps around the loop if L is much smaller than T . For example, if T is 102 and L is 5, the tortoise reaches the loop after 102 steps. The hare reaches the loop after 51 steps, spends the next 50 steps (100 cells) running 20 laps around the loop, and then moves one more step (two cells) inside the loop. In this case, $H = 2$.

The next question is, “When will the hare catch the tortoise?” When the tortoise enters the loop, the hare is H cells ahead, as shown in Figure 3.13. Because the tortoise and hare are in a loop, you can also think of the hare as $L - H$ cells behind the tortoise. Because the hare moves two cells for every one that the tortoise moves, it gains one cell per step. That means the hare will catch the tortoise in $L - H$ more steps.

In Figure 3.13, $H = 4$ and $L = 5$, so the hare will catch the tortoise in $5 - 4 = 1$ more step when both animals meet at cell E.

This means that, at the point of collision, the tortoise will have moved $L - H$ cells into the loop. When the two animals meet, they are $L - (L - H) = H$ cells short of the beginning of the loop. This is Important Fact #2.

IMPORTANT FACT #2

When the hare catches the tortoise, the two animals are H cells short of the beginning of the loop.

Now, if you could move the tortoise H cells from the point of collision, the tortoise would be at the beginning of the loop, and you would know where the loop starts. Unfortunately, you don’t know the value of H , so you can’t simply move the tortoise that far.

However, you do know from Important Fact #1 that if the tortoise moves T cells around the loop, it will end up H cells ahead of where it started. In this case, it will end up at the start of the loop!

Unfortunately, you also don’t know the value of T , so you can’t simply move the tortoise that far either. However, if you start the hare at the beginning of the linked list and make it move only one cell at a time instead of two (it’s probably tired after running around in the loop for so long), it will also reach the start of the loop after it crosses T cells. That means the two will meet again after crossing T cells, when they will be at the start of the loop.

The following pseudocode shows the algorithm at a high level:

1. Start the tortoise moving through the list at one cell per step. Start the hare moving through the list at two cells per step.
2. If the hare finds a `null` link, then the list has no loop, so stop.
3. Otherwise, when the hare catches the tortoise, restart the hare at the beginning of the list, moving one cell per step this time. Continue moving the tortoise at one cell per step.
4. When the tortoise and hare meet again, they are at the start of the loop. Leave the hare at the loop's starting point to take a well-deserved rest while the tortoise takes one more lap around the loop. When the tortoise's `Next` pointer gives the cell where the hare is waiting, the tortoise is at the end of the loop.
5. To break the loop, set the tortoise's cell's `Next` pointer to `null`.

WARNING I've never met a program that really needed to use the tortoise-and-hare algorithm. If you're careful, there's no excuse for letting a linked list become corrupted by an accidental loop. However, detecting loops seems to be a popular interview question and brainteaser, so it's good to know about this solution.

Loops in Doubly Linked Lists

Detecting loops in a doubly linked list is easy. If there is a loop, somewhere a `Next` pointer jumps back to an earlier part of the list. The `Prev` pointer in that cell points to an earlier cell, not the one that created the loop.

So, to detect a loop, simply traverse the list, and for each cell, verify that `cell.Next.Prev == cell`.

This all assumes that the cells form a normal doubly linked list and that a loop, if it exists, is a simple loop. If the `Next` and `Prev` lists are completely out of sync, this method detects the mess but doesn't help you fix it. This is more of a case of two threads through the same cells than a doubly linked list with a loop.

Summary

This chapter explained linked lists and some of the things that you can do with them. It explained singly and doubly linked lists and threaded lists. It also explained basic list-manipulation algorithms such as adding, finding, and deleting items. Finally, it described some more advanced algorithms to manage self-organizing lists and to detect and remove loops.